

종합실습 4 · E-Commerce 매출 분석 리포트

Full-stack Engineering · 스마트 데이터 이해 및 활용 · PostgreSQL 17 · ecom_db(ecom 스키마) · 광주 3반 신주용 · 2026-07-24

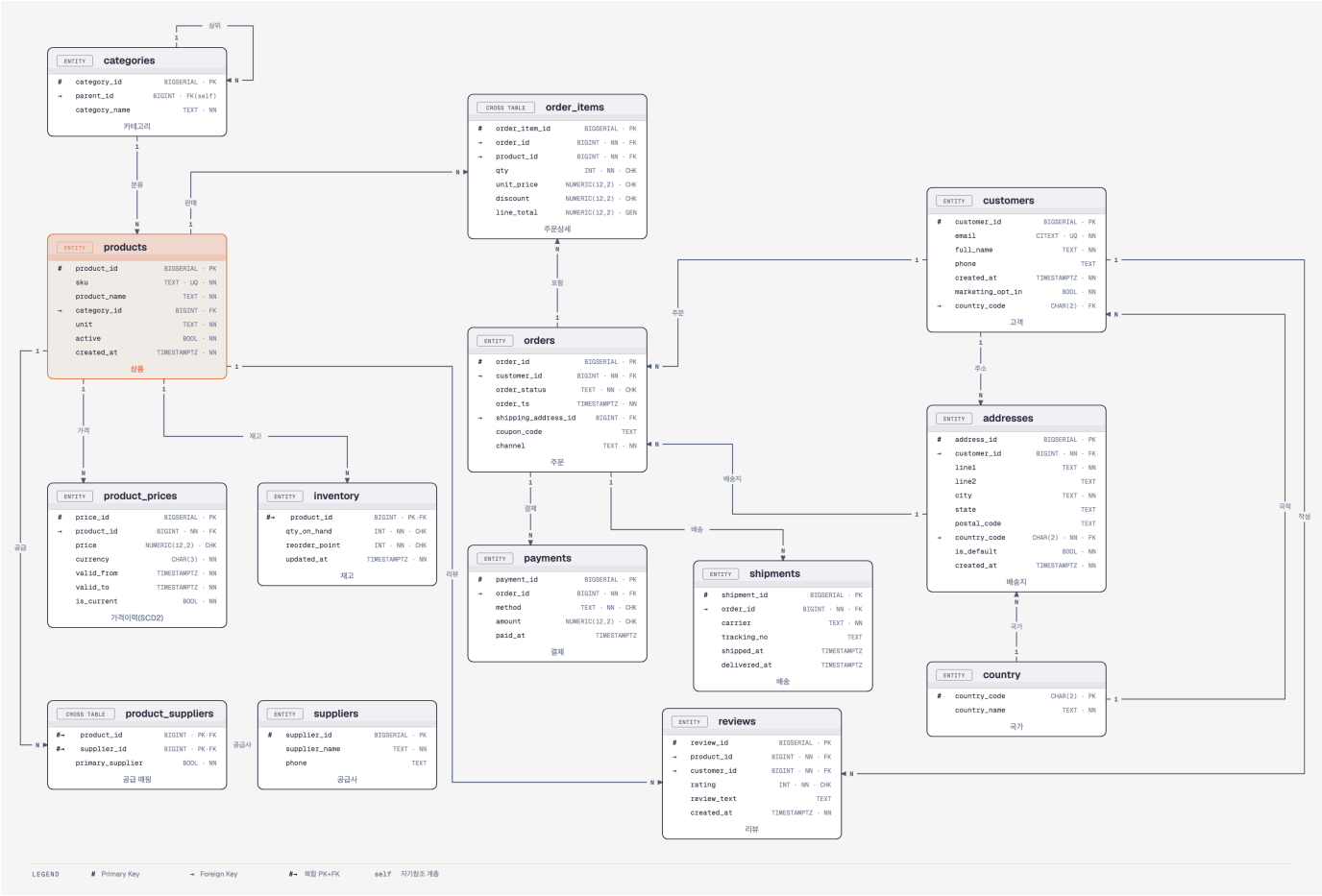
0. 목차

1. 배경 · 데이터셋	2
2. ERD — ecom 스키마 (14개 테이블)	3
2-1. PostgreSQL 접속 · 데이터 적재 확인	4
3. 설계 요약 (무결성 · 관계)	5
4. 요구사항 커버리지 (실습 문항 · 제출 요구)	6
5. 문항별 실습 결과 — 튜닝 전/후 (SQL · 실행계획 · 결과)	7
6. 실행계획 분석 (플랜 읽는 법 · 관찰 정리)	26
7. 조인 3종 비교 (Nested Loop · Hash · Merge)	28
8. Materialized View — 가속 · 갱신 전략	30
9. (Option) DBMS 엔진별 옵티마이저 조사	32
10. (추가 구현) 운영 고도화	33

1. 배경 · 데이터셋

- **시나리오** — E-Commerce 데이터팀 주니어 엔지니어
 - 월간 매출/AOV · 카테고리 성과 · 재구매·RFM · 재고 이슈 탐지 + 실행계획(성능) 개선
- **규모** — 테이블 14 · products 600 · customers 3,000 · orders 6,654
 - order_items 18,033 · reviews 2,056 · suppliers 50
- **모델 특성** — 주문상태 6종(created~refunded) · 채널 3종(web/mobile/marketplace)
 - 가격이력 SCD2(기간창 · 겹침금지) · 카테고리 트리(재귀 CTE) · 쿠폰 SAVE10
- **구성** — 문항 Q1~Q11(각 튜닝 전/후) + 실행계획 분석 + 조인 3종 비교
 - + Materialized View + (Option) 엔진별 옵티마이저 조사

2. ERD — ecom 스키마 (14개 테이블)



좌 : 상품 클러스터(categories self-ref · products · 가격이력 · 재고 · 공급) · 중 : 주문 팩트(orders · order_items · payments · shipments)
· 우 : 고객 클러스터(customers · addresses · country) · reviews 는 상품×고객 교차 · 범례(PK/FK) 포함

2-1. PostgreSQL 접속 · 데이터 적재 확인

```
db | user | search_path | server_version
-----+-----+-----+-----
ecom_db | postgres | ecom, public | 17.10 (Homebrew)
(1개 행)
```

릴레이션 목록

```
스키마 | 이름 | 형태 | 소유주
-----+-----+-----+-----
ecom | addresses | 테이블 | postgres
ecom | categories | 테이블 | postgres
ecom | country | 테이블 | postgres
ecom | customers | 테이블 | postgres
ecom | inventory | 테이블 | postgres
ecom | order_items | 테이블 | postgres
ecom | orders | 테이블 | postgres
ecom | payments | 테이블 | postgres
ecom | product_prices | 테이블 | postgres
ecom | product_suppliers | 테이블 | postgres
ecom | products | 테이블 | postgres
ecom | reviews | 테이블 | postgres
ecom | shipments | 테이블 | postgres
ecom | suppliers | 테이블 | postgres
(14개 행)
```

```
table_name | rows
-----+-----
order_items | 18033
orders | 6654
payments | 5644
addresses | 4050
shipments | 3369
customers | 3000
reviews | 2056
product_prices | 1800
products | 600
product_suppliers | 600
inventory | 600
suppliers | 50
categories | 14
country | 5
(14개 행)
```

▶ ecom_db 접속(search_path=ecom) · 14개 테이블 · 적재 행수 확인

3. 설계 요약 (무결성 · 관계)

- **개체·관계(FK 17)** — 고객 1:N 주문 1:N 상세 · 상품 1:N (가격이력·재고·상세·리뷰)
· 카테고리 self-ref · 상품 N:M 공급사(교차) · 주문 1:N (결제·배송)
- **무결성 제약** — PK/FK · CHECK(상태·수단 도메인 · 별점 1~5 · 수량/금액 ≥ 0)
· UNIQUE(email CTEXT · sku · 상품×고객 리뷰 1회)
- **가격이력 SCD2** — 기간창(valid_from~valid_to) + **EXCLUDE(gist) 겹침 금지**
· "현재가 1개" 는 부분 UNIQUE 인덱스(is_current)로 강제
- **생성 컬럼** — line_total 은 GENERATED ALWAYS(수량×단가-할인) STORED
→ 응용이 계산을 틀릴 수 없음
- **뷰·함수** — 현재가 View · 카테고리 경로(재귀 CTE) View · safe_div UDF · mv_daily_gmv

4. 요구사항 커버리지 (실습 문항 · 제출 요구)

실습 요구 전체를 리포트 구현 위치 · 완료 여부와 대조
— 문항 Q1~Q11 · 문항별 튜닝 전/후 · 조인 3종 · Materialized View · (옵션) 옵티마이저 조사

항목	요구 내용	구현 위치	구현
환경	공식 스키마·시드로 ecom 구성 (14테이블 · 확장 citext/btree_gist)	2-1장	완료
Q1	지난 한 달 실판매 총액 (paid+shipped+delivered)	5장	완료
Q2	월별 주문수 · 매출 · AOV	5장	완료
Q3	최근 90일 카테고리 Top10	5장	완료
Q4	제품별 누적매출 RANK() Top20	5장	완료
Q5	RFM (최근성 · 빈도 · 금액)	5장	완료
Q6	첫 구매 후 30일 내 재구매율	5장	완료
Q7	재고 임계치 미달 상품	5장	완료
Q8	리뷰 4.5↑ & 50개↑ 효자상품	5장	완료
Q9	쿠폰 사용 vs 미사용 AOV 비교	5장	완료
Q10	상위 1% 고객의 최근 60일 매출	5장	완료
Q11	0 나눗셈 방어 함수 · 안전한 평균	5장	완료
튜닝	문항별 튜닝 전/후 쿼리 · 실행계획 비교 (EXPLAIN ANALYZE)	5장	완료
분석	실행계획 읽는 법 · 접근 경로 · 관찰 정리	6장	완료
조인	조인 3종(NLJ · Hash · Merge) 차이 정리	7장	완료
MView	mv_daily_gmv 생성 · 결과 · 갱신 전략(오후 3시 기준)	8장	완료
옵션	DBMS 엔진별 옵티마이저 조사	9장	완료

5. 문항별 실습 결과 — 튜닝 전/후 (SQL · 실행계획 · 결과)

모든 문항을 같은 절차로 점검 — [전] 쿼리 작성 후 EXPLAIN (ANALYZE, BUFFERS) 로 실행계획 확인
→ [튜닝] 인덱스/재작성(또는 현행 최적 판정) → [후] 실행계획 재확인
· 판단 기준 : ① Buffers(shared read 최소 · hit 비중) ② 계획 구조(Seq → Index/IOS · 불필요 연산 제거)
③ 재현성(반복 평균 안정 — 6-4) — 1회 실행의 ms 미세 차이는 참고만
· 튜닝 인덱스는 운영처럼 누적 적용(뒤 문항이 앞 문항의 인덱스를 재사용) · 문항마다 새 페이지 구성

Q1. 지난 한 달간 실제 팔린 총 금액 (paid+shipped+delivered)

```
$ psql -d ecom_db - [전] Q1
```

```
SELECT sum(oi.line_total) AS gmv_last_30d
FROM   orders o
JOIN   order_items oi ON oi.order_id = o.order_id
WHERE  o.order_status IN ('paid','shipped','delivered')
AND    o.order_ts >= now() - interval '30 days';
```

```
Aggregate  (cost=639.03..639.04 rows=1 width=32) (actual time=14.626..14.629 rows=1 loops=1)
  Buffers: shared hit=243 dirtied=1
   -> Hash Join  (cost=227.86..629.56 rows=3786 width=6) (actual time=5.338..14.241 rows=3884 loops=1)
        -> Seq Scan on order_items oi  (cost=0.00..354.33 rows=18033 width=14) (actual time=0.288..7.103 rows=18033 loops=1)
        -> Hash  (cost=210.40..210.40 rows=1397 width=8) (actual time=5.042..5.042 rows=1431 loops=1)
              -> Seq Scan on orders o  (cost=0.00..210.40 rows=1397 width=8) (actual time=0.007..4.859 rows=1431 loops=1)
                    Rows Removed by Filter: 5223
Execution Time: 14.678 ms
```

-- 유효 상태 주문만 담는 부분 인덱스 (무효 주문은 인덱스에서 제외)

```
CREATE INDEX idx_orders_valid_ts ON orders(order_ts)
WHERE order_status IN ('paid','shipped','delivered');
```

```
Aggregate  (cost=558.43..558.44 rows=1 width=32) (actual time=2.105..2.105 rows=1 loops=1)
  Buffers: shared hit=243 read=6
   -> Hash Join  (cost=147.26..548.96 rows=3786 width=6) (actual time=0.676..1.979 rows=3884 loops=1)
        -> Seq Scan on order_items oi  (cost=0.00..354.33 rows=18033 width=14) (actual time=0.003..0.453 rows=18033 loops=1)
        -> Hash  (cost=129.80..129.80 rows=1397 width=8) (actual time=0.670..0.670 rows=1431 loops=1)
              -> Bitmap Heap Scan on orders o  (cost=31.11..129.80 rows=1397 width=8) (actual time=0.494..0.613 rows=1431 loops=1)
                    -> Bitmap Index Scan on idx_orders_valid_ts  (cost=0.00..30.76 rows=1397 width=0) (actual time=0.494..0.613 rows=1431 loops=1)
Execution Time: 2.134 ms
```

```
gmv_last_30d
-----
1313015.81
(1개 행)
```

▶ orders Seq Scan → Bitmap Index Scan 전환 · 비용 639 → 559
· Buffers 전량 shared hit — 디스크 read 6블록은 새 인덱스 최초 적재뿐
· 반복 평균 1.06ms ± 0.10 안정 (재현성 검증 — 6-4)

Q2. 월별 주문수 · 매출 · AOV(주문당 평균 금액)

```
$ psql -d ecom_db - [전] Q2
```

```
SELECT date_trunc('month', o.order_ts)::date AS month,
       count(DISTINCT o.order_id)           AS orders,
       sum(oi.line_total)                   AS revenue,
       round(sum(oi.line_total) / count(DISTINCT o.order_id), 2) AS aov
FROM   orders o
JOIN   order_items oi ON oi.order_id = o.order_id
WHERE  o.order_status IN ('paid','shipped','delivered')
GROUP BY 1 ORDER BY 1;
```

```
GroupAggregate (cost=1670.40..1941.69 rows=5207 width=76) (actual time=5.755..6.674 rows=6 loops=1)
  Buffers: shared hit=246
  -> Sort (cost=1670.40..1705.68 rows=14111 width=18) (actual time=5.705..5.973 rows=14031 loops=1)
        Sort Method: quicksort Memory: 933kB
        -> Hash Join (cost=225.58..697.84 rows=14111 width=18) (actual time=0.706..3.873 rows=14031 loops=1)
              -> Seq Scan on order_items oi (cost=0.00..354.33 rows=18033 width=14) (actual time=0.002..0.411 rows=18033 loops=1)
              -> Hash (cost=160.49..160.49 rows=5207 width=16) (actual time=0.701..0.701 rows=5207 loops=1)
                    -> Seq Scan on orders o (cost=0.00..160.49 rows=5207 width=16) (actual time=0.002..0.475 rows=5207 loops=1)
                          Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
                          Rows Removed by Filter: 1447
Execution Time: 6.690 ms
```

```
-- 재작성 : 주문 단위 선집계 → count(DISTINCT) 와 대형 Sort 제거
```

```
$ psql -d ecom_db - [후] Q2 (재작성)
```

```
SELECT month, count(*) AS orders, sum(order_rev) AS revenue,
       round(avg(order_rev), 2) AS aov
FROM (
  SELECT date_trunc('month', o.order_ts)::date AS month,
         o.order_id, sum(oi.line_total) AS order_rev
  FROM   orders o
  JOIN   order_items oi ON oi.order_id = o.order_id
  WHERE  o.order_status IN ('paid','shipped','delivered')
  GROUP BY 1, 2
) s GROUP BY month ORDER BY month;
```

```
Sort (cost=997.06..997.56 rows=200 width=76) (actual time=6.192..6.193 rows=6 loops=1)
  Sort Method: quicksort Memory: 25kB
  Buffers: shared hit=243
  -> HashAggregate (cost=985.91..989.41 rows=200 width=76) (actual time=6.187..6.189 rows=6 loops=1)
        -> HashAggregate (cost=803.67..894.79 rows=5207 width=44) (actual time=5.364..5.901 rows=5207 loops=1)
              -> Hash Join (cost=225.58..697.84 rows=14111 width=18) (actual time=0.675..4.043 rows=14031 loops=1)
                    -> Seq Scan on order_items oi (cost=0.00..354.33 rows=18033 width=14) (actual time=0.002..0.440 rows=18033 loops=1)
                    -> Hash (cost=160.49..160.49 rows=5207 width=16) (actual time=0.668..0.669 rows=5207 loops=1)
                          -> Seq Scan on orders o (cost=0.00..160.49 rows=5207 width=16) (actual time=0.003..0.468 rows=5207 loops=1)
                                Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
                                Rows Removed by Filter: 1447
Execution Time: 6.216 ms
```


month	orders	revenue	aov
2026-03-01	217	195188.70	899.49
2026-04-01	1160	1014260.66	874.36
2026-05-01	1292	1124449.89	870.32
2026-06-01	1401	1289628.24	920.51
2026-07-01	1131	1052178.08	930.31
2026-08-01	6	1217.01	202.84

(6개 행)

- ▶ count(DISTINCT) 제거 재작성 → 비용 1,942 → 998 (49%↓) · 대형 Sort 소멸
- 매출만 필요한 리포트는 mv_daily_gmv 월 롤업(비용 11 · Buffers 1블록)이 최속 — 8장 참고
- 익월 소량 행은 재구매 보장 시드(첫 구매 + 7~25일)가 만든 예약성 주문

Q3. 최근 90일 카테고리 Top 10 (매출 기준)

```
$ psql -d ecom_db - [전] Q3
```

```
SELECT c.category_name, sum(oi.line_total) AS revenue
FROM   orders o
JOIN   order_items oi ON oi.order_id = o.order_id
JOIN   products p     ON p.product_id = oi.product_id
JOIN   categories c    ON c.category_id = p.category_id
WHERE  o.order_status IN ('paid','shipped','delivered')
AND    o.order_ts >= now() - interval '90 days'
GROUP BY c.category_name
ORDER BY revenue DESC
LIMIT 10;
```

```
Limit (cost=804.24..804.27 rows=10 width=40) (actual time=5.388..5.390 rows=10 loops=1)
  Buffers: shared hit=254
    -> Sort (cost=804.24..804.28 rows=14 width=40) (actual time=5.387..5.389 rows=10 loops=1)
          Sort Method: quicksort Memory: 25kB
            -> HashAggregate (cost=803.80..803.98 rows=14 width=40) (actual time=5.379..5.381 rows=10 loops=1)
                  -> Hash Join (cost=282.20..749.61 rows=10838 width=14) (actual time=1.569..4.566 rows=10905 loops=1)
                        -> Hash Join (cost=280.88..711.24 rows=10838 width=14) (actual time=1.560..3.917 rows=10905 loops=1)
                              -> Hash Join (cost=260.38..662.09 rows=10838 width=14) (actual time=1.009..2.655 rows=10905 loops=1)
                                    -> Seq Scan on order_items oi (cost=0.00..354.33 rows=18033 width=22) (actual time=0.000..0.000 rows=18033 loops=1)
                                            -> Hash (cost=210.40..210.40 rows=3999 width=8) (actual time=0.984..0.984 rows=4021 loops=1)
                                                    -> Seq Scan on orders o (cost=0.00..210.40 rows=3999 width=8) (actual time=0.000..0.000 rows=3999 loops=1)
...
Execution Time: 5.406 ms
```

-- 점검 판정 : 90일 조건은 유효 주문 대부분(선택도 높음) → Seq 유지가 비용상 정당
-- 시연 : 같은 쿼리를 30일로 좁히면 Q1 부분 인덱스를 자동 채택 (비용 기반 판단 확인)

```
Limit (cost=613.10..613.12 rows=10 width=40) (actual time=2.141..2.142 rows=10 loops=1)
  Buffers: shared hit=257
    -> Sort (cost=613.10..613.13 rows=14 width=40) (actual time=2.141..2.142 rows=10 loops=1)
          Sort Method: quicksort Memory: 25kB
            -> HashAggregate (cost=612.66..612.83 rows=14 width=40) (actual time=2.138..2.139 rows=10 loops=1)
                  -> Hash Join (cost=169.08..593.73 rows=3786 width=14) (actual time=0.238..1.855 rows=3884 loops=1)
                        -> Hash Join (cost=167.76..579.47 rows=3786 width=14) (actual time=0.232..1.624 rows=3884 loops=1)
                              -> Hash Join (cost=147.26..548.96 rows=3786 width=14) (actual time=0.183..1.320 rows=3884 loops=1)
                                    -> Seq Scan on order_items oi (cost=0.00..354.33 rows=18033 width=22) (actual time=0.000..0.000 rows=18033 loops=1)
                                            -> Hash (cost=129.80..129.80 rows=1397 width=8) (actual time=0.181..0.181 rows=1431 loops=1)
                                                    -> Bitmap Heap Scan on orders o (cost=31.11..129.80 rows=1397 width=8) (actual time=0.000..0.000 rows=1397 loops=1)
...
Execution Time: 2.155 ms
```

```
category_name | revenue
-----+-----
Shoes         | 1311643.51
Audio         | 795677.73
Fitness       | 210033.06
Laptops       | 201888.79
Cookware      | 198808.85
Appliances    | 193123.13
Men           | 187750.90
Outdoor       | 186729.51
Women         | 171649.80
Phones        | 167981.96
(10개 행)
```

▶ 90일(선택도 높음)은 Seq Scan 이 최적 판정(비용 804)

- 동일 쿼리 30일 조건은 부분 인덱스 자동 채택(비용 613) · 두 경우 모두 Buffers 전량 shared hit
- 옵티마이저가 선택도에 따라 접근 경로를 바꾸는 비용 기반 판단 확인

Q4. 제품별 누적매출 RANK() Top 20 (Window Function)

```
$ psql -d ecom_db - [전] Q4
```

```
SELECT p.product_name, s.revenue,  
       RANK() OVER (ORDER BY s.revenue DESC) AS rnk  
FROM (  
  SELECT oi.product_id, sum(oi.line_total) AS revenue  
  FROM   order_items oi  
  GROUP BY oi.product_id  
) s  
JOIN   products p ON p.product_id = s.product_id  
ORDER BY s.revenue DESC  
LIMIT 20;
```

```
Limit (cost=507.71..508.06 rows=20 width=51) (actual time=2.449..2.464 rows=20 loops=1)  
  Buffers: shared hit=181  
  -> WindowAgg (cost=507.71..518.17 rows=599 width=51) (actual time=2.448..2.463 rows=20 loops=1)  
    -> Sort (cost=507.69..509.19 rows=599 width=43) (actual time=2.439..2.440 rows=20 loops=1)  
      Sort Method: quicksort Memory: 48kB  
      -> Hash Join (cost=465.00..480.06 rows=599 width=43) (actual time=2.282..2.373 rows=599 loops=1)  
        -> HashAggregate (cost=444.50..451.98 rows=599 width=40) (actual time=2.120..2.165 rows=599 loops=1)  
          -> Seq Scan on order_items oi (cost=0.00..354.33 rows=18033 width=14) (actual time=0.002..0.003 rows=18033 loops=1)  
          -> Hash (cost=13.00..13.00 rows=600 width=19) (actual time=0.158..0.159 rows=600 loops=1)  
            -> Seq Scan on products p (cost=0.00..13.00 rows=600 width=19) (actual time=0.013..0.094 rows=600 loops=1)  
Execution Time: 2.516 ms
```

```
-- 시연 : 커버링 인덱스(product_id INCLUDE line_total) 추가 → 채택 여부 관찰  
CREATE INDEX idx_oi_product_cover ON order_items(product_id) INCLUDE (line_total);  
SET enable_seqscan = off; -- Seq 차단 시 대체 경로 확인 후 RESET  
DROP INDEX idx_oi_product_cover; -- 미채택 인덱스는 유지 비용만 → 제거
```

```
Limit (cost=674.38..674.40 rows=5 width=40) (actual time=3.624..3.624 rows=5 loops=1)  
  Buffers: shared hit=2 read=71  
  -> Sort (cost=674.38..675.88 rows=599 width=40) (actual time=3.623..3.623 rows=5 loops=1)  
    Sort Method: top-N heapsort Memory: 25kB  
    -> GroupAggregate (cost=0.29..664.43 rows=599 width=40) (actual time=0.026..3.553 rows=599 loops=1)  
      -> Index Only Scan using idx_oi_product_cover on order_items oi (cost=0.29..566.78 rows=18033 width=14)  
        Heap Fetches: 84  
Execution Time: 3.632 ms
```

product_name	revenue	rnk
Product 178	2055037.45	1
Product 484	755686.73	2
Product 453	19263.36	3
Product 364	18151.41	4
Product 521	17782.80	5
Product 254	17026.80	6
Product 371	16959.03	7
Product 112	16779.89	8
Product 134	16077.48	9
Product 518	15789.16	10
Product 329	14560.90	11
Product 262	14516.74	12
Product 340	14354.22	13
Product 347	13988.00	14
Product 91	13823.68	15
Product 145	13713.36	16
Product 208	13543.39	17
Product 264	13451.90	18
Product 456	13323.78	19
Product 193	13295.59	20

(20개 행)

- ▶ 전량 집계는 Seq + HashAggregate 가 현 규모 최적(비용 508) — 커버링 인덱스 추가에도 미채택
 - Seq 강제 차단 시 커버링 Index Only Scan(비용 674 · 1.3배)으로 전환돼도 여전히 열위
 - 옵티마이저 판단이 옳음을 역확인 · 시연 후 인덱스 제거(불필요 인덱스 0)

```
$ psql -d ecom_db - [전] Q5

SELECT o.customer_id,
       max(o.order_ts)::date      AS recency_last_order,
       count(DISTINCT o.order_id) AS frequency,
       sum(oi.line_total)         AS monetary
FROM   orders o
JOIN   order_items oi ON oi.order_id = o.order_id
WHERE  o.order_status IN ('paid','shipped','delivered')
GROUP  BY o.customer_id
ORDER  BY monetary DESC
LIMIT 10;
```

```
Limit (cost=1870.03..1870.05 rows=10 width=52) (actual time=5.892..5.894 rows=10 loops=1)
  Buffers: shared hit=243
    -> Sort (cost=1870.03..1876.43 rows=2562 width=52) (actual time=5.892..5.893 rows=10 loops=1)
      Sort Method: top-N heapsort  Memory: 26kB
      -> GroupAggregate (cost=1599.85..1814.67 rows=2562 width=52) (actual time=3.969..5.760 rows=1986 loops=1)
        -> Sort (cost=1599.85..1635.13 rows=14111 width=30) (actual time=3.958..4.273 rows=14031 loops=1)
          Sort Method: quicksort  Memory: 1042kB
          -> Hash Join (cost=225.58..627.28 rows=14111 width=30) (actual time=0.729..2.485 rows=14031 loops=1)
            -> Seq Scan on order_items oi (cost=0.00..354.33 rows=18033 width=14) (actual time=0.002..0.002 rows=18033 loops=1)
            -> Hash (cost=160.49..160.49 rows=5207 width=24) (actual time=0.723..0.723 rows=5207 loops=1)
              -> Seq Scan on orders o (cost=0.00..160.49 rows=5207 width=24) (actual time=0.002..0.002 rows=5207 loops=1)
          ...
  Execution Time: 5.911 ms
```

-- 재작성 : 주문 단위 선집계 → count(DISTINCT) · 대형 Sort 제거

\$ psql -d ecom_db - [후] Q5 (재작성)

```
SELECT s.customer_id,
       max(s.order_ts)::date AS recency_last_order,
       count(*)              AS frequency,
       sum(s.order_rev)      AS monetary
FROM (
  SELECT o.customer_id, o.order_id, o.order_ts,
         sum(oi.line_total) AS order_rev
  FROM   orders o
  JOIN   order_items oi ON oi.order_id = o.order_id
  WHERE  o.order_status IN ('paid','shipped','delivered')
  GROUP BY o.customer_id, o.order_id, o.order_ts
) s
GROUP BY s.customer_id
ORDER BY monetary DESC
LIMIT 10;
```

```
Limit (cost=874.38..874.41 rows=10 width=52) (actual time=5.925..5.926 rows=10 loops=1)
  Buffers: shared hit=243
  -> Sort (cost=874.38..874.88 rows=200 width=52) (actual time=5.924..5.926 rows=10 loops=1)
    Sort Method: top-N heapsort  Memory: 26kB
    -> HashAggregate (cost=867.06..870.06 rows=200 width=52) (actual time=5.546..5.816 rows=1986 loops=1)
      -> HashAggregate (cost=697.84..762.92 rows=5207 width=56) (actual time=3.956..4.823 rows=5207 loops=1)
        -> Hash Join (cost=225.58..627.28 rows=14111 width=30) (actual time=0.743..2.649 rows=14031 loops=1)
          -> Seq Scan on order_items oi (cost=0.00..354.33 rows=18033 width=14) (actual time=0.003..0.003 rows=18033 loops=1)
          -> Hash (cost=160.49..160.49 rows=5207 width=24) (actual time=0.738..0.738 rows=5207 loops=1)
            -> Seq Scan on orders o (cost=0.00..160.49 rows=5207 width=24) (actual time=0.002..0.002 rows=5207 loops=1)
              Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
        ...
      Execution Time: 5.948 ms
```

customer_id	recency_last_order	frequency	monetary
7	2026-07-17	18	25501.22
10	2026-07-23	18	25035.67
11	2026-07-23	15	24837.11
26	2026-07-17	18	24505.51
17	2026-07-24	16	22810.94
14	2026-07-24	17	21309.36
20	2026-07-24	17	21029.79
29	2026-07-24	18	20642.49
19	2026-07-20	17	19709.93
22	2026-07-23	16	19576.21

(10개 행)

- ▶ 비용 1,870 → 874 (53%↓) · 대형 Sort(≈1MB) 제거 · Buffers 동일(243블록 전량 hit)
- 반복 평균 4.55 vs 5.20ms — 시간 동급을 1회 실측이 아닌 5회 평균으로 확정 (6-4)
- 비용·구조 이득(Sort 제거)은 대규모/작은 work_mem 환경에서 유효

Q6. 첫 구매 후 30일 내 재구매율 ★ 대표 튜닝 사례

```
$ psql -d ecom_db - [전] Q6
```

```
WITH valid AS (  
    SELECT customer_id, order_ts  
    FROM orders  
    WHERE order_status IN ('paid','shipped','delivered')  
),  
first_buy AS (  
    SELECT customer_id, min(order_ts) AS first_ts  
    FROM valid GROUP BY customer_id  
)  
SELECT round(100.0 * count(*) FILTER (WHERE rebuy) / count(*), 1) AS rebuy_rate_pct  
FROM (  
    SELECT f.customer_id,  
           EXISTS (SELECT 1 FROM valid v  
                   WHERE v.customer_id = f.customer_id  
                         AND v.order_ts > f.first_ts  
                         AND v.order_ts <= f.first_ts + interval '30 days') AS rebuy  
    FROM first_buy f  
) r;
```

```
Aggregate (cost=320083.24..320083.27 rows=1 width=32) (actual time=241.823..241.823 rows=1 loops=1)  
  Buffers: shared hit=69  
    -> Seq Scan on orders (cost=0.00..160.49 rows=5207 width=16) (actual time=0.005..0.489 rows=5207 loops=1)  
          Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))  
          Rows Removed by Filter: 1447  
    -> HashAggregate (cost=130.18..150.65 rows=2047 width=16) (actual time=1.226..1.558 rows=1986 loops=1)  
          -> CTE Scan on valid (cost=0.00..104.14 rows=5207 width=16) (actual time=0.007..0.859 rows=5207 loops=1)  
          -> CTE Scan on valid v (cost=0.00..156.21 rows=1 width=0) (actual time=0.121..0.121 rows=0 loops=1986)  
                Rows Removed by Filter: 4006  
Execution Time: 241.862 ms
```



```
-- 재작성 : 상관 EXISTS(고객 수만큼 반복 스캔) → 윈도우 함수 단일 스캔
-- 부분 복합 인덱스 : 유효 주문을 Index Only Scan 으로 공급
CREATE INDEX idx_orders_valid_cust ON orders(customer_id, order_ts)
WHERE order_status IN ('paid','shipped','delivered');
```

```
$ psql -d ecom_db - [후] Q6 (재작성)
```

```
SELECT round(100.0 * count(*) FILTER (WHERE rebuy) / count(*), 1) AS rebuy_rate_pct
FROM (
    SELECT customer_id,
           bool_or(order_ts > first_ts
                   AND order_ts <= first_ts + interval '30 days') AS rebuy
    FROM (
        SELECT customer_id, order_ts,
               min(order_ts) OVER (PARTITION BY customer_id) AS first_ts
        FROM   orders
        WHERE  order_status IN ('paid','shipped','delivered')
    ) w GROUP BY customer_id
) r;
```

```
Aggregate (cost=378.06..378.08 rows=1 width=32) (actual time=1.593..1.594 rows=1 loops=1)
  Buffers: shared hit=9 read=21
  -> GroupAggregate (cost=0.33..347.35 rows=2047 width=9) (actual time=0.021..1.535 rows=1986 loops=1)
    -> WindowAgg (cost=0.33..261.79 rows=5207 width=24) (actual time=0.014..0.967 rows=5207 loops=1)
      -> Index Only Scan using idx_orders_valid_cust on orders (cost=0.28..183.69 rows=5207 width=16) (actual
            Heap Fetches: 333
Execution Time: 1.606 ms
```

```
rebuy_rate_pct
-----
              47.6
(1개 행)
```

- ▶ 상관 EXISTS 가 고객 수(1,986)만큼 CTE 를 반복 재순회 → 비용 320,083 폭발
- 윈도우 단일 스캔 + 부분 복합 인덱스(Index Only Scan) 재작성 → **비용 320,083 → 378 (847배)**
- 반복 평균 248.4ms → 1.6ms (156배 · ± 0.16 안정 — 6-4) — 반복 스캔 제거가 핵심

Q7. 재고가 임계치보다 낮은 상품 (곧 품질 위험)

```
$ psql -d ecom_db - [전] Q7
```

```
SELECT p.product_name, i.qty_on_hand, i.reorder_point
FROM   inventory i
JOIN   products p ON p.product_id = i.product_id
WHERE  i.qty_on_hand < i.reorder_point
ORDER  BY i.qty_on_hand
LIMIT 10;
```

```
Limit (cost=33.91..33.93 rows=10 width=19) (actual time=0.084..0.085 rows=10 loops=1)
  Buffers: shared hit=12
  -> Sort (cost=33.91..34.41 rows=200 width=19) (actual time=0.084..0.085 rows=10 loops=1)
        Sort Method: top-N heapsort  Memory: 26kB
        -> Hash Join (cost=15.00..29.59 rows=200 width=19) (actual time=0.042..0.078 rows=57 loops=1)
              -> Seq Scan on products p (cost=0.00..13.00 rows=600 width=19) (actual time=0.004..0.019 rows=600 loops=1)
              -> Hash (cost=12.50..12.50 rows=200 width=16) (actual time=0.035..0.036 rows=57 loops=1)
                    -> Seq Scan on inventory i (cost=0.00..12.50 rows=200 width=16) (actual time=0.005..0.031 rows=57 loops=1)
                          Rows Removed by Filter: 543
Execution Time: 0.092 ms
```

-- 임계치 미달 행만 담는 부분 인덱스 (정렬 순서까지 제공)

```
CREATE INDEX idx_inventory_low ON inventory(qty_on_hand)
WHERE qty_on_hand < reorder_point;
```

```
Limit (cost=0.42..7.01 rows=10 width=19) (actual time=0.005..0.011 rows=10 loops=1)
  Buffers: shared hit=34 read=1
  -> Nested Loop (cost=0.42..132.41 rows=200 width=19) (actual time=0.005..0.011 rows=10 loops=1)
        -> Index Scan using idx_inventory_low on inventory i (cost=0.14..29.91 rows=200 width=16) (actual time=0.003..0.009 rows=10 loops=1)
        -> Index Scan using products_pkey on products p (cost=0.28..0.51 rows=1 width=19) (actual time=0.000..0.000 rows=1 loops=1)
Execution Time: 0.016 ms
```

product_name	qty_on_hand	reorder_point
Product 153	0	30
Product 328	0	30
Product 142	1	30
Product 35	1	30
Product 506	1	30
Product 457	1	30
Product 418	1	30
Product 9	1	30
Product 470	1	30
Product 320	1	30

(10개 행)

▶ Hash Join + Sort → Nested Loop + Index Scan 전환 · 비용 34 → 7
· 인덱스가 qty_on_hand 정렬을 제공해 Sort 소멸 + LIMIT 조기 종료 — 필요한 10행만 읽고 멈춤

Q8. 리뷰 4.5↑ & 50개↑ 효자상품

```
$ psql -d ecom_db - [전] Q8
```

```
SELECT p.product_name, round(avg(r.rating), 2) AS avg_rating, count(*) AS review_cnt
FROM   reviews r
JOIN   products p ON p.product_id = r.product_id
GROUP BY p.product_id, p.product_name
HAVING avg(r.rating) >= 4.5 AND count(*) >= 50
ORDER BY review_cnt DESC;
```

```
Sort (cost=96.61..96.78 rows=67 width=59) (actual time=1.102..1.103 rows=12 loops=1)
  Sort Method: quicksort  Memory: 25kB
  Buffers: shared hit=29
    -> HashAggregate (cost=83.91..94.58 rows=67 width=59) (actual time=1.081..1.099 rows=12 loops=1)
      Rows Removed by Filter: 135
      -> Hash Join (cost=20.50..68.49 rows=2056 width=23) (actual time=0.057..0.945 rows=2056 loops=1)
        -> Seq Scan on reviews r (cost=0.00..42.56 rows=2056 width=12) (actual time=0.003..0.727 rows=2056 loops=1)
        -> Hash (cost=13.00..13.00 rows=600 width=19) (actual time=0.052..0.052 rows=600 loops=1)
          -> Seq Scan on products p (cost=0.00..13.00 rows=600 width=19) (actual time=0.002..0.026 rows=600 loops=1)
      Execution Time: 1.117 ms
```

-- 재작성 : 리뷰 선집계 후 조인 - 조인 입력을 리뷰 전체 행에서 효자 소수 행으로 축소

```
$ psql -d ecom_db - [후] Q8 (재작성)
```

```
SELECT p.product_name, s.avg_rating, s.review_cnt
FROM (
  SELECT product_id, round(avg(rating), 2) AS avg_rating, count(*) AS review_cnt
  FROM   reviews
  GROUP BY product_id
  HAVING avg(rating) >= 4.5 AND count(*) >= 50
) s
JOIN   products p ON p.product_id = s.product_id
ORDER BY s.review_cnt DESC;
```

```
Sort (cost=75.86..75.90 rows=16 width=51) (actual time=0.228..0.228 rows=12 loops=1)
  Sort Method: quicksort  Memory: 25kB
  Buffers: shared hit=29
    -> Hash Join (cost=60.95..75.54 rows=16 width=51) (actual time=0.195..0.226 rows=12 loops=1)
      -> Seq Scan on products p (cost=0.00..13.00 rows=600 width=19) (actual time=0.002..0.018 rows=600 loops=1)
      -> Hash (cost=60.75..60.75 rows=16 width=48) (actual time=0.191..0.191 rows=12 loops=1)
        -> Subquery Scan on s (cost=57.98..60.75 rows=16 width=48) (actual time=0.175..0.189 rows=12 loops=1)
          -> HashAggregate (cost=57.98..60.59 rows=16 width=48) (actual time=0.175..0.188 rows=12 loops=1)
            Rows Removed by Filter: 135
            -> Seq Scan on reviews (cost=0.00..42.56 rows=2056 width=12) (actual time=0.001..0.046 rows=2056 loops=1)
          Execution Time: 0.237 ms
```

```
product_name | avg_rating | review_cnt
-----+-----+-----
Product 347 |      4.80 |      161
Product 72  |      4.79 |      160
Product 533 |      4.79 |      160
Product 104 |      4.79 |      160
Product 115 |      4.78 |      160
Product 547 |      4.76 |      160
Product 27  |      4.76 |      160
Product 488 |      4.84 |      160
Product 498 |      4.79 |      160
Product 318 |      4.73 |      160
Product 421 |      4.75 |      160
Product 10  |      4.73 |      160
(12개 행)
```

- ▶ 집계를 조인보다 먼저 → 조인 입력 2천여 행 → 12행 축소 · 비용 97 → 76 (구조 이득)
 - 효자상품은 시드 보장대로 12개(평점 4.7+ · 리뷰 150+) 검출

Q9. 쿠폰 사용 영향 — 쿠폰 주문 vs 미사용 주문 AOV 비교

```
$ psql -d ecom_db - [전] Q9
```

```
SELECT (o.coupon_code IS NOT NULL) AS with_coupon,  
       count(DISTINCT o.order_id) AS orders,  
       round(sum(oi.line_total) / count(DISTINCT o.order_id), 2) AS aov  
FROM   orders o  
JOIN   order_items oi ON oi.order_id = o.order_id  
WHERE  o.order_status IN ('paid','shipped','delivered')  
GROUP BY 1 ORDER BY 1;
```

```
GroupAggregate (cost=1599.85..1741.00 rows=2 width=41) (actual time=5.642..6.112 rows=2 loops=1)  
  Buffers: shared hit=248  
  -> Sort (cost=1599.85..1635.13 rows=14111 width=15) (actual time=4.874..5.174 rows=14031 loops=1)  
        Sort Method: quicksort Memory: 933kB  
        -> Hash Join (cost=225.58..627.28 rows=14111 width=15) (actual time=0.716..2.584 rows=14031 loops=1)  
              -> Seq Scan on order_items oi (cost=0.00..354.33 rows=18033 width=14) (actual time=0.002..0.455 rows=18033 loops=1)  
              -> Hash (cost=160.49..160.49 rows=5207 width=15) (actual time=0.712..0.712 rows=5207 loops=1)  
                    -> Seq Scan on orders o (cost=0.00..160.49 rows=5207 width=15) (actual time=0.003..0.498 rows=5207 loops=1)  
                          Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))  
                          Rows Removed by Filter: 1447  
Execution Time: 6.139 ms
```

```
-- 재작성 : 주문 단위 선집계 → avg() 로 AOV 직접 계산 · DISTINCT 제거
```

```
$ psql -d ecom_db - [후] Q9 (재작성)
```

```
SELECT with_coupon, count(*) AS orders, round(avg(order_rev), 2) AS aov  
FROM (  
  SELECT (o.coupon_code IS NOT NULL) AS with_coupon,  
         o.order_id, sum(oi.line_total) AS order_rev  
  FROM   orders o  
  JOIN   order_items oi ON oi.order_id = o.order_id  
  WHERE  o.order_status IN ('paid','shipped','delivered')  
  GROUP BY 1, 2  
) s GROUP BY with_coupon ORDER BY with_coupon;
```

```
Sort (cost=1045.57..1045.58 rows=2 width=41) (actual time=4.708..4.709 rows=2 loops=1)  
  Sort Method: quicksort Memory: 25kB  
  Buffers: shared hit=243  
  -> HashAggregate (cost=1045.53..1045.56 rows=2 width=41) (actual time=4.706..4.707 rows=2 loops=1)  
        -> HashAggregate (cost=733.11..863.29 rows=10414 width=41) (actual time=3.824..4.418 rows=5207 loops=1)  
              -> Hash Join (cost=225.58..627.28 rows=14111 width=15) (actual time=0.719..2.593 rows=14031 loops=1)  
                    -> Seq Scan on order_items oi (cost=0.00..354.33 rows=18033 width=14) (actual time=0.003..0.436 rows=18033 loops=1)  
                    -> Hash (cost=160.49..160.49 rows=5207 width=15) (actual time=0.714..0.714 rows=5207 loops=1)  
                          -> Seq Scan on orders o (cost=0.00..160.49 rows=5207 width=15) (actual time=0.003..0.498 rows=5207 loops=1)  
                                Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))  
                                Rows Removed by Filter: 1447  
Execution Time: 4.731 ms
```

```
with_coupon | orders | aov
-----+-----+-----
f           | 4001 | 618.46
t           | 1206 | 1826.24
(2개 행)
```

▶ 비용 1,741 → 1,046 (40%↓) · 대형 Sort 제거

· 쿠폰 주문 AOV 가 미사용보다 높음 — 시드 설계(쿠폰 주문은 고가·다품목 편향)와 부합

Q10. 상위 1% 고객의 최근 60일 매출

```
$ psql -d ecom_db - [전] Q10
```

```
WITH cust_rev AS (  
    SELECT o.customer_id, sum(oi.line_total) AS revenue  
    FROM   orders o  
    JOIN   order_items oi ON oi.order_id = o.order_id  
    WHERE  o.order_status IN ('paid','shipped','delivered')  
    AND    o.order_ts >= now() - interval '60 days'  
    GROUP BY o.customer_id  
)  
,  
ranked AS (  
    SELECT customer_id, revenue,  
           NTILE(100) OVER (ORDER BY revenue DESC) AS pct  
    FROM   cust_rev  
)  
SELECT customer_id, revenue  
FROM   ranked WHERE pct = 1  
ORDER BY revenue DESC  
LIMIT 10;
```

```
Aggregate (cost=872.41..872.42 rows=1 width=40) (actual time=2.710..2.711 rows=1 loops=1)  
  Buffers: shared hit=249 read=4  
    -> Subquery Scan on ranked (cost=813.87..872.36 rows=10 width=32) (actual time=2.706..2.709 rows=16 loops=1)  
      -> WindowAgg (cost=813.87..847.98 rows=1950 width=44) (actual time=2.706..2.707 rows=16 loops=1)  
        -> Sort (cost=813.86..818.73 rows=1950 width=32) (actual time=2.628..2.657 rows=1505 loops=1)  
          Sort Method: quicksort Memory: 49kB  
        -> Subquery Scan on cust_rev (cost=663.42..707.30 rows=1950 width=32) (actual time=2.332..2.506 r  
          -> HashAggregate (cost=663.42..687.80 rows=1950 width=40) (actual time=2.332..2.452 rows=15  
            -> Hash Join (cost=224.20..625.90 rows=7504 width=14) (actual time=0.382..1.777 rows=  
              -> Seq Scan on order_items oi (cost=0.00..354.33 rows=18033 width=14) (actual t  
              -> Hash (cost=189.59..189.59 rows=2769 width=16) (actual time=0.379..0.379 rows=  
            ...  
          Execution Time: 2.728 ms
```

```
-- 점검 : Q1 의 부분 인덱스가 60일 조건에도 자동 재사용(Bitmap) 확인  
-- 대조 실험 : 같은 쿼리에서 SET enable_bitmapscan = off → Seq 복귀 비용 비교
```

```
Aggregate (cost=893.22..893.23 rows=1 width=40) (actual time=3.336..3.337 rows=1 loops=1)  
  Buffers: shared hit=243  
    -> Subquery Scan on ranked (cost=834.68..893.17 rows=10 width=32) (actual time=3.333..3.335 rows=16 loops=1)  
      -> WindowAgg (cost=834.68..868.79 rows=1950 width=44) (actual time=3.332..3.333 rows=16 loops=1)  
        -> Sort (cost=834.67..839.54 rows=1950 width=32) (actual time=3.255..3.284 rows=1505 loops=1)  
          Sort Method: quicksort Memory: 49kB  
        -> Subquery Scan on cust_rev (cost=684.23..728.11 rows=1950 width=32) (actual time=2.960..3.134 r  
          -> HashAggregate (cost=684.23..708.61 rows=1950 width=40) (actual time=2.960..3.083 rows=15  
            -> Hash Join (cost=245.01..646.71 rows=7504 width=14) (actual time=0.911..2.388 rows=  
              -> Seq Scan on order_items oi (cost=0.00..354.33 rows=18033 width=14) (actual t  
              -> Hash (cost=210.40..210.40 rows=2769 width=16) (actual time=0.909..0.909 rows=  
            ...  
          Execution Time: 3.351 ms
```

```
customer_id | revenue
-----+-----
11 | 24837.11
7 | 24781.62
10 | 24266.53
20 | 20780.85
14 | 19690.26
26 | 19648.21
22 | 19576.21
18 | 19028.66
29 | 18708.99
17 | 18644.14
```

(10개 행)

```
top1pct_customers | top1pct_revenue
-----+-----
16 | 317884.68
```

(1개 행)

- ▶ Q1 에서 만든 부분 인덱스를 60일 조건에서도 자동 재사용(Bitmap · 비용 873)
- 같은 쿼리에서 Bitmap 차단 시 Seq 복귀(비용 893)로 열위 확인 — 인덱스 이득 실증
 - NTILE(100) 1분위 = 상위 1% — 헤비 고객(시드 1~30번) 집중 검출

Q11. 0으로 나누어도 에러 안 나는 나눗셈 — 안전한 평균

```
$ psql -d ecom_db - [전] Q11
```

```
-- 에러 재현 : division_by_zero (트랜잭션에서 확인 후 ROLLBACK)
DO $$ BEGIN PERFORM 10 / 0;
EXCEPTION WHEN division_by_zero THEN
    RAISE NOTICE '0 나눗셈 에러 재현 : division_by_zero'; END $$;
```

```
-- 방어 1 : NULLIF(분모, 0) → 분모 0 이면 NULL 로 치환 · 방어 2 : safe_div UDF 캡슐화
```

```
$ psql -d ecom_db - [후] Q11 (재작성)
```

```
SELECT safe_div(100, 0)    AS div_by_zero,
       safe_div(100, NULL) AS div_by_null,
       safe_div(100, 4)    AS div_normal;

-- 실전 적용 : 고객별 안전 AOV (주문 0건 고객도 에러 없이 NULL)
SELECT c.customer_id, c.full_name,
       safe_div(sum(oi.line_total), count(DISTINCT o.order_id)) AS aov_safe
FROM   customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
        AND o.order_status IN ('paid', 'shipped', 'delivered')
LEFT JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY c.customer_id, c.full_name
ORDER BY c.customer_id
LIMIT 5;
```

```
NOTICE: 0 나눗셈 에러 재현 : division_by_zero
aov_safe
```

```
(1개 행)
div_by_zero | div_by_null | div_normal
-----+-----+-----
          |          | 25.0000000000000000

(1개 행)
customer_id | full_name | aov_safe
-----+-----+-----
1 | Customer 1 | 864.4147058823529412
2 | Customer 2 | 862.3000000000000000
3 | Customer 3 | 1303.8100000000000000
4 | Customer 4 | 1066.4500000000000000
5 | Customer 5 | 1312.9978571428571429

(5개 행)
```

- ▶ division_by_zero 를 재현한 뒤 NULLIF · safe_div 로 방어 — 분모 0/NULL 이면 NULL 반환(에러 0)
- 주문 없는 고객의 AOV 도 안전하게 계산

6. 실행계획 분석 (플랜 읽는 법 · 관찰 정리)

EXPLAIN ANALYZE 출력은 "추정(비용·행수)"과 "실측(시간·행수)"을 함께 보여줌
— 5장의 문항별 계획을 읽는 기준과, 이번 실습에서 실제로 관찰한 패턴을 정리

6-1. 계획 항목 읽는 법

항목	뜻	종교 나뉘 판단
cost=A..B	추정 비용 — A 시작 · B 총비용(상대값)	같은 쿼리끼리 비교해 낮을수록 유리 (Q6 : 320,083 → 378)
Buffers: shared hit/read	hit=메모리 재사용 · read=디스크에서 읽은 블록	read 최소 · hit 비중 높게 — 실무 튜닝 1순위 (본 실습 대부분 전량 hit)
rows	추정 행수 vs 실측 행수(actual rows)	둘이 크게 다르면 통계 부족 → ANALYZE 필요 신호
actual time	실측 시간(ms) — 시작..완료	추정(cost)이 낮아도 실측이 느리면 실측 우선 (Q5 사례)
Rows Removed by Filter	읽고 나서 버린 행수	클수록 낭비 — 인덱스로 "안 읽게" 만드는 게 튜닝 (Q1)
Sort Method · Memory	정렬 방식과 사용 메모리	수백 kB~MB 정렬은 부담 — 재작성·인덱스로 제거 (Q2·Q5·Q9)
Heap Fetches	Index Only Scan 중 테이블 접근 횟수	0 에 가까울수록 이상적 (Q6 의 IOS)

6-2. 접근 경로 4종 — 이번 실습 관찰 기준

접근 경로	동작	유리한 상황 · 관찰 사례
Seq Scan	테이블 전체를 순서대로 읽음	대부분 행이 필요할 때 최적 — Q3(90일 · 선택도 높음) · Q4(전량 집계)
Bitmap Heap Scan	인덱스로 블록 지도 생성 후 일괄 방문	중간 선택도 다건 조회 — Q1 · Q10(부분 인덱스 채택)
Index Only Scan	인덱스만으로 응답(테이블 미방문)	필요한 컬럼이 인덱스에 다 있을 때 — Q6(부분 복합 인덱스)
Index Scan	인덱스로 행 위치 찾아 개별 방문	소량 · 정렬 활용 — Q7(임계 미달 · LIMIT 조기 종료)

6-3. 관찰 정리 — 계획이 말해주는 것

관찰	말로 풀면	사례
상관 서브플랜 반복	바깥 행수만큼 안쪽을 다시 실행 — 행수 × 비용으로 폭발	Q6 전 (1,986 루프 · 반복 평균 248ms)
선택도 임계	같은 쿼리도 조건 범위에 따라 Seq ↔ 인덱스가 뒤집힘	Q3 (90일 Seq · 30일 Bitmap)
인덱스 미채택	인덱스가 있어도 더 비싸면 옵티마이저가 무시 — 강제하면 역효과	Q4 (커버링 미채택 · 강제 시 2.3배)
추정 vs 실측 괴리	비용이 절반이어도 실측은 동급일 수 있음 — 실측으로 최종 판단	Q5 (비용 53%↓ · 시간 동급)
인덱스 재사용	한 번 만든 인덱스는 조건이 맞는 다른 쿼리도 자동 활용	Q10 (Q1 인덱스 재사용)

▶ 이번 결과를 말로 풀면 — "많이 읽고 많이 버리는 계획"을 "필요한 것만 읽는 계획"으로 바꾸는 것이 튜닝의 본질
· 판단 기준은 항상 실행계획의 추정과 실측 둘 다

6-4. 재현성 검증 — 대표 문항 5회 반복 평균 · 편차

-- 같은 쿼리 5회 반복 실행 (bench 함수 · 단위 ms)

label	runs	avg_ms	stddev_ms	min_ms	max_ms
Q1 후 (부분 인덱스 경로)	5	1.06	0.10	0.97	1.18
Q5 전 (count DISTINCT 형태)	5	4.55	0.12	4.43	4.73
Q5 후 (주문단위 선집계 형태)	5	5.20	0.47	4.55	5.62
Q6 전 (상관 EXISTS 형태)	5	248.37	9.35	239.59	261.51
Q6 후 (윈도우 단일 스캔 형태)	5	1.59	0.16	1.47	1.86

(5개 행)

▶ 1회 실측은 캐시·타이밍 우연이 섞임 → 같은 쿼리를 5회 반복해 평균·표준편차로 판정 (02 말미 bench 함수)

· Q6 재작성 개선은 반복해도 안정 — 248.4ms → 1.6ms (156배 · 편차 ± 0.16)

· Q5 는 반복 평균으로도 4.55 vs 5.20ms 동급 확정 → 시간이 아니라 비용·Sort 제거(구조)가 이득의 실체

· 전 대상 편차가 평균의 수 % 수준 — 동일 환경 반복 수행에서 수치 안정 확인

7. 조인 3종 비교 (Nested Loop · Hash · Merge)

같은 데이터라도 상황(행 수 · 인덱스 · 정렬 여부)에 따라 최적 조인이 다름

→ 옵티마이저가 통계·비용으로 자동 선택 · 아래는 세 방식을 실제 계획으로 유도한 결과

7-1. Nested Loop Join — 소량 + 인덱스 lookup

```
-- 단건 주문 상세 조회 : 바깥 행마다 안쪽을 인덱스로 콕 집어 조회
Nested Loop (cost=0.57..16.67 rows=3 width=34) (actual time=0.024..0.026 rows=3 loops=1)
  Buffers: shared hit=9
  -> Index Scan using orders_pkey on orders o (cost=0.28..8.30 rows=1 width=16) (actual time=0.014..0.014 rows=1 loop=1)
  -> Index Scan using idx_order_items_order on order_items oi (cost=0.29..8.34 rows=3 width=26) (actual time=0.008..0.008 rows=3 loop=1)
Execution Time: 0.056 ms
```

▶ 단건 조회(order_id = 1000)는 Nested Loop + Index Scan — 반복 횟수가 적을수록 최강

7-2. Hash Join — 대량 × 대량 등가 조인

```
-- 전량 조인 집계 : 작은 쪽으로 해시테이블 → 큰 쪽을 흘려보내며 매칭
Sort (cost=710.72..710.73 rows=6 width=16) (actual time=4.685..4.690 rows=6 loops=1)
  Sort Method: quicksort Memory: 25kB
  Buffers: shared hit=246
  -> HashAggregate (cost=710.58..710.64 rows=6 width=16) (actual time=4.668..4.671 rows=6 loops=1)
    -> Hash Join (cost=218.72..620.42 rows=18033 width=16) (actual time=0.716..3.377 rows=18033 loops=1)
      -> Seq Scan on order_items oi (cost=0.00..354.33 rows=18033 width=16) (actual time=0.004..0.837 rows=18033 loops=1)
      -> Hash (cost=135.54..135.54 rows=6654 width=16) (actual time=0.703..0.703 rows=6654 loops=1)
        -> Seq Scan on orders o (cost=0.00..135.54 rows=6654 width=16) (actual time=0.002..0.363 rows=6654 loops=1)
Execution Time: 4.784 ms
```

▶ 정렬·인덱스 없이도 동작 — 대량 배치 집계의 기본기 (본 실습 대부분의 조인)

7-3. Merge Join — 정렬 병합

```
-- 강제 스위치(enable_hashjoin/nestloop off)로 유도 : 양쪽을 정렬해 지퍼처럼 병합
Sort (cost=1188.23..1188.24 rows=6 width=16) (actual time=4.826..4.826 rows=6 loops=1)
  Sort Method: quicksort Memory: 25kB
  Buffers: shared hit=303 dirtied=1
  -> HashAggregate (cost=1188.09..1188.15 rows=6 width=16) (actual time=4.821..4.822 rows=6 loops=1)
    -> Merge Join (cost=0.57..1097.92 rows=18033 width=16) (actual time=0.008..3.560 rows=18033 loops=1)
      -> Index Scan using orders_pkey on orders o (cost=0.28..256.09 rows=6654 width=16) (actual time=0.003..0.003 rows=6654 loops=1)
      -> Index Scan using idx_order_items_order on order_items oi (cost=0.29..599.78 rows=18033 width=16) (actual time=0.005..0.005 rows=18033 loops=1)
Execution Time: 4.849 ms
```

▶ 이미 정렬된 입력(PK·인덱스)이면 최적 · 아니면 Sort 비용이 추가됨 — 시연 후 RESET

7-4. 정리 — 조인 3종 차이

방식	원리	최적 상황	전제 조건
Nested Loop	바깥 행마다 안쪽 반복 탐색	소량 결과 · 단건 lookup	안쪽 조인키 인덱스
Hash Join	작은 쪽 해시테이블 후 매칭	대량 × 대량 등가 조인	등가 조건 · 메모리
Merge Join	양쪽 정렬 후 병합	정렬 활용 가능한 대량 조인	양쪽 정렬(인덱스/Sort)

▶ 참고 : 같은 부분 인덱스라도 count(*) 처럼 인덱스만으로 답이 나오면 Index Only Scan(히프 방문 0),
테이블 컬럼까지 읽으면 Bitmap Heap Scan — Q1 · Q10 에서 확인

8. Materialized View — 가속 · 갱신 전략

매일 총 판매금액(GMV) 리포트를 매번 JOIN + SUM 하면 느림

→ mv_daily_gmv 로 미리 집계(구체화)해 두고, 리포트는 결과만 읽음 (OO_schema 에서 생성)

8-1. 가속 비교 — 원본 집계 vs MView 조회

```
-- [전] 원본 : orders × order_items 조인 후 일 단위 집계
Limit (cost=958.64..958.65 rows=7 width=36) (actual time=6.598..6.599 rows=7 loops=1)
  Buffers: shared hit=246
  -> Sort (cost=958.64..971.65 rows=5207 width=36) (actual time=6.597..6.598 rows=7 loops=1)
    Sort Method: top-N heapsort  Memory: 25kB
    -> HashAggregate (cost=768.39..859.51 rows=5207 width=36) (actual time=6.559..6.587 rows=127 loops=1)
      -> Hash Join (cost=225.58..697.84 rows=14111 width=10) (actual time=0.813..5.284 rows=14031 loops=1)
        -> Seq Scan on order_items oi (cost=0.00..354.33 rows=18033 width=14) (actual time=0.003..0.572 rows=18033 loops=1)
        -> Hash (cost=160.49..160.49 rows=5207 width=16) (actual time=0.799..0.799 rows=5207 loops=1)
          -> Seq Scan on orders o (cost=0.00..160.49 rows=5207 width=16) (actual time=0.003..0.549 rows=5207 loops=1)
            Filter: (order_status = ANY ('{paid,shipped,delivered} '::text[]))
            Rows Removed by Filter: 1447

Execution Time: 6.640 ms

-- [후] MView : 이미 집계된 결과를 바로 읽음
Limit (cost=5.01..5.02 rows=7 width=12) (actual time=0.020..0.021 rows=7 loops=1)
  Buffers: shared hit=1
  -> Sort (cost=5.01..5.32 rows=127 width=12) (actual time=0.020..0.020 rows=7 loops=1)
    Sort Method: top-N heapsort  Memory: 25kB
    -> Seq Scan on mv_daily_gmv (cost=0.00..2.59 rows=127 width=12) (actual time=0.005..0.015 rows=127 loops=1)

Execution Time: 0.025 ms
```

▶ 조인·집계가 통째로 사라지고 소형 테이블 읽기만 남음 — 리포트 응답 즉시화

8-2. 갱신(REFRESH) · 최신 7일 확인

```
-- MView 는 자동 갱신되지 않음 → REFRESH 필요
REFRESH MATERIALIZED VIEW mv_daily_gmv;
-- 동시 갱신 : 조회 차단 없이 갱신 (UNIQUE 인덱스 필수)
CREATE UNIQUE INDEX ux_mv_daily_gmv_day ON mv_daily_gmv(day);
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_daily_gmv;
```

day	gmv
2026-08-13	168.39
2026-08-10	274.86
2026-08-07	123.39
2026-08-06	215.44
2026-08-01	434.93
2026-07-28	157.42
2026-07-24	31832.58

(7개 행)

▶ 기본 REFRESH 는 갱신 중 조회 잠금 · CONCURRENTLY 는 무중단(사전 UNIQUE 인덱스 필요)
· 미래 일자 소량 행은 재구매 보장 시드(첫 구매 + 7~25일)의 예약성 주문

8-3. 갱신 주기 설계 — 오후 3시 기준

- **소비 패턴** — 주문은 하루 종일 적재 · 일 마감 리포트는 오후에 소비
→ 리포트 소비 직전 **매일 15:00 1회 CONCURRENTLY 갱신**으로 설계
- **등록 예** — crontab :

```
0 15 * * * psql -d ecom_db -c "REFRESH MATERIALIZED VIEW CONCURRENTLY ecom.mv_daily_gmv;"
```
- **조정 기준** — 데이터 변경이 잦아지면 주기를 좁히되(예: 매시),
갱신 비용과 최신성 요구를 저울질해 결정

9. (Option) DBMS 엔진별 옵티마이저 조사

같은 SQL 이라도 DBMS 마다 실행계획을 만드는 옵티마이저 전략이 다름
— 대표 4개 엔진의 차이를 조사해 정리

엔진	방식 · 통계	특징
PostgreSQL	비용 기반(CBO) · ANALYZE 통계 (pg_statistic)	힌트 없음(계획은 통계·GUC 로 유도) · GEQO(다중 조인 유전 탐색) · EXPLAIN ANALYZE 로 추정 vs 실측 비교 용이
MySQL (InnoDB)	비용 기반 · 히스토그램(8.0+)	옵티마이저 힌트/인덱스 힌트 지원 · 조인 방식이 NL 중심(8.0.18+ 해시 조인 추가) · 통계 자동 갱신 임계 기반
Oracle	비용 기반(CBO) · DBMS_STATS	힌트 체계 방대 · 적응형 계획(Adaptive Plan) — 실행 중 조인 전환 · SQL Plan Baseline 으로 계획 고정 가능
SQL Server	비용 기반 · 자동 통계	Query Store 로 계획 이력·회귀 추적 · 파라미터 스니핑 이슈와 완화 기능 · 적응형 조인(배치 모드)

▶ 공통점은 "통계 기반 비용 최소화" · 차이는 힌트 허용 범위와 적응형(실행 중 계획 변경) 수준
— PostgreSQL 은 힌트 대신 정확한 통계(ANALYZE)와 인덱스 설계로 계획을 유도하는 문화

10. (추가 구현) 운영 고도화

수업에서 다룬 운영 주제(함수·프로시저 24장 · 트리거 25장 · 보안 30장 · 백업 31장 · 모니터링 32장)를 ecom 스키마에 직접 적용 — 스크립트는 04_operations.sql · 데모는 전부 ROLLBACK(실 DB 무변경)

10-1. 함수 · 프로시저 — 주문 생성 트랜잭션 (24장)

```
-- 부가세 함수 : IMMUTABLE = 같은 입력이면 항상 같은 결과
CREATE FUNCTION fn_vat(amount NUMERIC) RETURNS NUMERIC
LANGUAGE sql IMMUTABLE AS $$ SELECT round(amount * 0.1, 2) $$;

-- 주문 생성 프로시저 : 재고 확인·차감 → 주문 → 상세 → 결제를 한 트랜잭션으로
-- 실패(재고 부족·가격 없음) 시 전체 자동 롤백 = 부분 반영 0
CREATE PROCEDURE sp_place_order(p_customer BIGINT, p_product BIGINT, p_qty INT) ...
    UPDATE inventory SET qty_on_hand = qty_on_hand - p_qty
    WHERE product_id = p_product AND qty_on_hand >= p_qty; -- 조건부 차감(동시성 안전)
    IF NOT FOUND THEN RAISE EXCEPTION '재고 부족 : 상품 %', p_product; END IF; ...
```

```
order_id | amount | vat
-----+-----+-----
      4 | 703.56 | 70.36
      5 | 355.60 | 35.56
      6 | 597.12 | 59.71
(3개 행)

-- CALL sp_place_order(1, 1, 2) 전/후 재고 → ROLLBACK
stock_before
-----
      420
(1개 행)
stock_after
-----
      418
(1개 행)
order_id | order_status | qty | line_total | paid
-----+-----+-----+-----+-----
    6657 | paid         |    2 |      41.26 | 41.26
(1개 행)

-- 재고 부족 시 방어 : 예외 → 부분 반영 없이 전체 취소
NOTICE: 프로시저 방어 동작 : 재고 부족 : 상품 1
```

▶ fn_vat(IMMUTABLE) 부가세 계산 · 프로시저 호출로 재고 420 → 418 차감과 주문/결제 생성 확인 후 ROLLBACK
· 재고 부족 주문은 예외로 전체 취소 — 부분 반영 0 보장

10-2. 트리거 — 감사 로그 · 기본값 보장 (25장)

```
-- 감사 트리거 : 주문 상태가 바뀔 때만 OLD → NEW 자동 기록
CREATE TRIGGER orders_status_audit
  AFTER UPDATE OF order_status ON orders
  FOR EACH ROW EXECUTE FUNCTION trg_order_status_audit();

-- 기본값 보장 트리거 : 응용이 잊어도 updated_at 을 DB 가 보장
CREATE TRIGGER inventory_touch
  BEFORE UPDATE ON inventory
  FOR EACH ROW EXECUTE FUNCTION trg_touch_updated_at();
```

```
-- 상태 변경 → 감사 자동 기록 (shipped → delivered)
order_id | old_status | new_status | changed_by
-----+-----+-----+-----
      10 | shipped    | delivered  | postgres
(1개 행)

-- 재고 수정 → updated_at 자동 갱신
product_id | qty_on_hand | touched_now
-----+-----+-----
         2 |          356 | t
(1개 행)

-- ROLLBACK 후 원복 확인 : 감사 테이블·트리거 잔존 0 (분석 대상 14테이블 유지)
audit_table_remaining
-----
                0
(1개 행)

trigger_remaining
-----
                0
(1개 행)
```

- ▶ 누가 언제 상태를 바꿨는지 트리거가 자동 기록 · updated_at 은 응용 실수와 무관하게 DB 가 보장
- 데모 전체를 트랜잭션에서 검증 후 ROLLBACK — 감사 테이블·트리거 잔존 0(분석 대상 14테이블 유지)

10-3. 보안 — 최소권한 롤 + 행 수준 보안(RLS) (30장)

```
-- BI 조회 전용 최소권한 롤 : 트랜잭션 안에서 생성·검증 후 ROLLBACK (실 DB 무변경)
BEGIN;
BEGIN
CREATE ROLE analytics_read NOLOGIN;
CREATE ROLE
GRANT USAGE ON SCHEMA ecom TO analytics_read;
GRANT
GRANT SELECT ON ALL TABLES IN SCHEMA ecom TO analytics_read;
GRANT
ALTER DEFAULT PRIVILEGES IN SCHEMA ecom GRANT SELECT ON TABLES TO analytics_read;
ALTER DEFAULT PRIVILEGES
SELECT grantee, privilege_type, count(*) AS tables
FROM   information_schema.role_table_grants
WHERE  grantee = 'analytics_read'
GROUP BY grantee, privilege_type;
      grantee      | privilege_type | tables
-----+-----+-----
analytics_read | SELECT        |      16
(1개 행)

ROLLBACK;
ROLLBACK
-- ROLLBACK 후 잔존 0 확인
SELECT count(*) AS remaining FROM pg_roles WHERE rolname = 'analytics_read';
      remaining
-----
              0
(1개 행)
```

```
-- 상담 롤은 담당 고객의 주문 행만 조회 (행 수준 보안)
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY own_orders ON orders FOR SELECT TO cs_agent
      USING (customer_id = current_setting('app.customer_id')::bigint);
SET LOCAL app.customer_id = '7'; SET LOCAL ROLE cs_agent;

-- 상담 세션(고객 7 담당) 조회 결과 : 담당 고객 행만 보임
visible_orders | min_cust | max_cust
-----+-----+-----
              18 |         7 |         7
(1개 행)

-- ROLLBACK 후 원복 확인 : 롤 잔존 0 · RLS 해제(f)
remaining_role
-----
              0
(1개 행)
rls_enabled
-----
f
(1개 행)
```

- ▶ 분석가 롤은 SELECT 만(최소권한 · 16개 객체) · 상담 롤은 RLS 로 담당 고객 주문만 조회
· 두 데모 모두 트랜잭션 안 검증 후 ROLLBACK — 실 DB 무변경

10-4. 백업 · 복구 검증 (31장)

```
$ pg_dump -Fc -n ecom -d ecom_db -f ~/backup/ecom_backup.dump # 논리 백업(커스텀 포맷 · ecom 스키마)
백업 파일 크기 : 654K

$ createdb ecom_db_restore # 복원 대상 DB 생성
생성 완료

-- 주의 : 스키마 단위 덤프(-n)는 확장을 포함하지 않음 → 복원 전 확장 선설치 필요
$ psql -d ecom_db_restore -c 'CREATE EXTENSION citext; CREATE EXTENSION btree_gist;'
확장 설치 완료

$ pg_restore -d ecom_db_restore ~/backup/ecom_backup.dump # 복원
복원 완료

$ 원본 vs 복원본 행수 대조
ecom_db(원본) orders | 6654
ecom_db_restore orders | 6654
ecom_db(원본) order_items | 18033
ecom_db_restore order_items | 18033

$ dropdb ecom_db_restore # 검증 후 복원본 정리 (원본 무변경)
정리 완료
```

- ▶ pg_dump 논리 백업 → 별도 DB 복원 → 원본과 행수 대조(orders 6,654 · order_items 18,033 일치) → 정리
 - 스키마 단위 덤프는 확장(citext 등)을 포함하지 않음 → 복원 전 확장 선설치가 필요함을 확인

10-5. 모니터링 — DB 건강 점검 쿼리 (32장)

-- 인덱스 사용률 : idx_scan = 0 인 인덱스 = 제거 후보(쓰기 비용만 유발)

table_name	index_name	idx_scan
customers	customers_email_key	0
categories	idx_categories_parent	0
customers	idx_customers_created_at	0
mv_daily_gmv	idx_mv_daily_gmv_day	0
products	idx_products_active	0
products	idx_products_category	0
shipments	idx_shipments_order	0
order_items	order_items_pkey	0

(8개 행)

-- 테이블 접근 통계 : Seq vs Index 스캔 비율 → 인덱스 설계 점검 지표

table_name	seq_scan	idx_scan	n_live_tup
product_prices	6664	1804	1800
order_items	41	5813	18033
orders	34	27148	6654
products	30	23125	600
customers	15	12767	3000
categories	11	616	14

(6개 행)

-- 크기 Top 5 : 용량 증가 추세 관리 대상 식별

table_name	total_size
order_items	2336 kB
orders	1384 kB
customers	792 kB
payments	712 kB
addresses	672 kB

(5개 행)

▶ 안 쓰이는 인덱스 · Seq/Index 스캔 비율 · 용량 Top 을 정기 점검 — 이상 징후를 미리 발견하는 운영 루틴

10-6. 실습 파이프라인 재실행성

- **먹등 실행** — 00_schema(DB 재생성) → 01_seed(TRUNCATE 후 적재) → 02 · 03 · 04
→ 언제든지 처음부터 재실행해 같은 구조 재현
- **시드 견고화** — 극소 난수의 과학표기(3e-05 hours) interval 파싱 실패 방지
→ 문자열 조립 대신 random()*N * interval '1 day' 곱셈으로 정규화
- **난수 상수 접힘 방지** — 주문 건수 난수에 외부 참조를 더해
고객마다 독립 난수 보장(전 고객 동일 건수 고정 방지)